

Class 6: R Functions

Amy Nguyen (PID: A18148284)

Table of contents

Background	1
Our first function	1
A second function	3
A Protein generating function	5

Background

All functions in R have at least 3 things

- A **name** that we use to call the function
- One or more input **arguments**
- The **body** the lines of R code that do the work

Our first function

Let's write a silly wee function called `add()` to add some numbers (the input arguments)

```
add <- function(x, y) {  
  x + y  
}
```

Now we can use this function

```
add(x=100, y=1)
```

```
[1] 101
```

```
add(x=10, y=10)
```

```
[1] 20
```

```
add(x=c(100, 1, 100), y=1)
```

```
[1] 101    2 101
```

Q. What if I give a multiple element vector to x and y?

```
add(x=c(100,1), y=c(100,1))
```

```
[1] 200    2
```

Q. What if I give three inputs to the function?

```
#add(x=c(100,1), y=1, z=1)
```

Q. What if I only give one input to the add function?

```
addnew <- function(x, y=1) {  
  x + y  
}
```

```
addnew(x=100)
```

```
[1] 101
```

```
addnew(c(100,1), 100)
```

```
[1] 200 101
```

If we write our function with input arguments having no default value then the user will be required to set them when they use the function. We can give our input arguments “default” values by setting them equal to some sensible value - e.g. y=1 in the `addnew()` function.

A second function

Let's try something more interesting: Make a sequence generating tool..

The `sample()` function can be a useful starting point here:

```
sample(1:10, size=4)
```

```
[1] 6 1 3 10
```

Q. Generate 9 random numbers taken from the input vector `x=1:10`?

```
sample(1:10, size=9)
```

```
[1] 9 2 1 4 7 8 10 3 5
```

Q. Generate 12 random numbers taken from the input vector `x=1:10`?

```
sample(1:10, size=12, replace = TRUE)
```

```
[1] 1 9 3 9 4 6 9 8 2 1 10 7
```

Q. Write code for the `sample()` function that generates nucleotide sequences of length 6?

```
sample(x=c("A","T","C","G"), size=6, replace = TRUE)
```

```
[1] "C" "T" "G" "A" "T" "G"
```

Q. Write a first function `generate_dna` that returns a user specified length of DNA sequence:

```
generate_dna <- function(len=6) {  
  sample(x=c("A","T","C","G"), size=len, replace = TRUE)  
}
```

```
generate_dna(len=100)
```

```

[1] "A" "A" "A" "G" "G" "T" "C" "T" "T" "C" "C" "T" "C" "C" "C" "T" "T" "T"
[19] "C" "T" "T" "C" "C" "T" "G" "A" "T" "G" "G" "A" "A" "G" "A" "C" "A" "T"
[37] "C" "A" "A" "A" "G" "A" "T" "G" "A" "A" "G" "G" "T" "G" "T" "A" "G" "A"
[55] "T" "C" "A" "T" "C" "T" "G" "T" "A" "A" "A" "A" "A" "T" "G" "G" "G" "A"
[73] "C" "A" "C" "G" "A" "G" "A" "T" "A" "A" "A" "A" "C" "T" "C" "T" "C" "G"
[91] "C" "A" "T" "G" "A" "C" "G" "C" "G" "A"

```

Key-Points Every function in R looks fundamentally the same in terms of its structure. Basically 3 things: name, input, and body

```

name <- function(input) {
  body
}

```

Functions can have multiple inputs. These can be **required** arguments or **optional** arguments. With optional arguments having a set default value.

Q. Modify and improve our `generate_dna()` function to return it's generated sequence in a more standard format like "AGTAGTA" rather than the vector "A", "C", "G", "A"

```

generate_dna <- function(len=6, fasta=TRUE) {

  ans <- sample(x=c("A","T","C","G"),
               size=len, replace = TRUE)
  if(fasta) {
    cat("Single-element vector output")
    ans <- paste(ans, collapse = "")
  } else {
    cat("Multi-element vector output") }
  return(ans)
}

generate_dna(fasta=FALSE)

```

Multi-element vector output

```

[1] "G" "A" "T" "A" "G" "G"

```

The `paste()` function - it's job is to join up or stick together (a.k.a paste) input strings together

```
paste("alice", "loves R", sep=" ")
```

```
[1] "alice loves R"
```

Flow control means where the R brain goes in your code

```
good_mood <- TRUE

if(good_mood) {
  cat("Great!")
} else {
  cat("Bummer!")
}
```

Great!

A Protein generating function

Q. Write a function, called `generate_protein` that generates a user specified length protein sequence.

```
generate_protein <- function(len)
# The amino-acids to sample from
{
  aa <- sample(x=c("A", "R", "N", "D", "C", "Q", "E", "G", "H", "I", "L", "K", "M", "F", "P", "S", "T", "W", "V"),
              size=len, replace = T)

  # Draw n=len amino acids to make our sequence
  ans <- sample(aa, size=len, replace = T)
  ans <- paste(ans, collapse = "")
  return(ans)
}
```

Q. Use that function to generate random protein sequences between length 6 and 12

```
generate_protein(6)
```

```
[1] "NFLGNH"
```

```
generate_protein(7)
```

```
[1] "IENYMYS"
```

```
generate_protein(8)
```

```
[1] "YNKAQDVC"
```

```
generate_protein(9)
```

```
[1] "YMCMGWGGM"
```

```
generate_protein(10)
```

```
[1] "NWARPWMLRV"
```

```
generate_protein(11)
```

```
[1] "CYRTDCWDDEI"
```

```
generate_protein(12)
```

```
[1] "TCDSAHAQEDQN"
```

```
for(i in 6:12) {  
  # FASTA ID line ">id"  
  cat(">", i, sep="", "\n")  
  # Our protein sequence line  
  cat( generate_protein(i), "\n")  
}
```

```
>6
```

```
SRLNRM
```

```
>7
```

```
RVGDMGQ
```

```
>8
```

ELATLNLS
>9
WPPFQGYCN
>10
AQTSKCWNSK
>11
DARIMWGGPRL
>12
ADRIAYWQMTCE

Q. Are any of your sequences unique i.e. not found anywhere in nature? My 9th, 10th, 11th, and 12th sequences were unique.